Shaheed Zulfikar Ali Bhutto Institute of Science & Technology

COMPUTER SCIENCE DEPARTMENT

Software Construction & Development Lab

Mids Report

Submitted To: Ma'am Kashia Riaz

Student Name: Abdul Rafay

Reg Number: 2280101

Section: BSSE 5A

"Online ATM System"

Introduction

The Online ATM System significantly enhances banking convenience by enabling users to carry out a wide range of financial transactions through a ATM machine. This innovative system offers secure and efficient financial management, allowing users to access their accounts and perform transactions anytime and anywhere, streamlining the banking experience beyond traditional interactions.

Main Features

Account Creation

- Users can create new bank accounts by providing personal information like name, age, city, and selecting account types (Savings or Current).
- An account number is generated randomly, and the initial balance is set as per the user's input. The account data (up to 100 accounts) is stored in an array.
- A unique 4-digit PIN is required for security, and the system checks to ensure that no two accounts share the same PIN.
- After creating an account, the system offers users the choice to continue to account operations or exit.

Account Operations

- **Withdraw:** Users can withdraw money from their account if their balance is sufficient. The remaining balance is displayed after every withdrawal.
- **Transfer:** Users can transfer money between accounts by entering the recipient's account number and transfer amount. The system verifies the recipient's account and updates balances accordingly.
- **Deposit:** Users can deposit a specified amount into their account, and the updated balance is shown.
- **Check Balance:** Users can view their current account balance at any time without modifying it. This allows users to keep track of their funds and monitor their account status.
- **Change PIN**: Users can change their PIN if they provide a unique new PIN not used by other accounts.
- **Transaction History**: Every transaction is recorded and can be viewed at any time, providing users with transparency regarding their account activity.
- **Block Card**: Users can block their ATM card in case of loss or theft to prevent unauthorized transactions. A confirmation message will be displayed once the card is blocked.
- **Unblock Card**: Users can unblock their ATM card if it has been previously blocked, given that they verify their identity, typically through a secure method such as answering security questions or entering the correct PIN.
- **Apply for Loan**: Users can initiate a loan application process, which includes providing necessary details like loan amount, purpose, and tenure. The system can guide them through the application steps and provide confirmation of submission.
- **Exit to Main Menu**: Users can return to the main menu from any transaction screen to choose a different action, ensuring they can easily navigate through the system without needing to start over.

User Input & Validation

- The system takes user input via the Scanner class and ensures valid inputs before processing any banking operations.
- Account numbers and PINs are checked to ensure secure access to account data.
- It validates this input to ensure only valid actions are performed, such as ensuring PINs are unique and sufficient funds exist before allowing withdrawals or transfers.

Description

1. Java Variables

Variables are used to store information; about the bank accounts, the user's input, and other temporary data needed to run the banking operations.

- Variables like firstName, lastName, city, and balance are used to store essential account holder details.
- Sensitive fields such as pin and accountNumber ensure secure user identification and account management.

2. Data Types

The choice of data types is critical for storing the right kind of information:

- double is used for balance and accountNumber because these values can be large and may include decimal points.
- int is used for pin and other options because PINs are whole numbers, and integers are efficient for such use cases.
- String is used for names (firstName and lastName), and Java's String class provides methods to manipulate and handle text data.
- Choosing the appropriate data type ensures that the operations performed on the data are efficient and accurate.

3. Typecasting (Widening)

Implicit typecasting (widening) occurs when integers (such as user input) are used in operations involving doubles (like balance), ensuring smooth operations without errors.

```
//widening
double accountNumber = Math.random() * 1000000000; // Generate random account number
```

4. Loops

Loops are crucial for managing the flow of the application, especially when users need to perform repeated operations.

while loop

- Another loop ensures that each user creates a unique PIN, asking them to re-enter until a unique PIN is chosen.

```
while (true) {
    System.out.print("Enter a unique 4-digit pin: ");
    pin = scanner.nextInt();

    // Check if the pin is already used
    if (!usedPins.contains(pin)) {
        usedPins.add(pin);
        break; // Exit loop if the PIN is unique
    } else {
        System.out.println("This PIN is already taken. Please choose a different PIN.");
    }
}
```

- while (true) loop in the main method keeps the ATM system operational, displaying options continuously until the user selects to exit.

```
while (true) {
    System.out.println("\nWELCOME TO THE ONLINE ATM SYSTEM");
    System.out.println("1: Create a new account");
    System.out.println("2: ATM operations");
    System.out.println("0: Exit");
    System.out.print("Please select an option (1-2, 0 to exit): ");
```

- Inside the ATM operations case, another while (true) loop allows a user to continue with multiple transactions until they choose to exit.

```
while (true) {
    System.out.println("\nSelect an Option:");
    System.out.println("1: Withdraw");
    System.out.println("2: Transfer");
    System.out.println("3: Deposit");
    System.out.println("4: Check Balance");
    System.out.println("5: Change PIN");
    System.out.println("6: View Transaction History");
    System.out.println("7: Block Card");
    System.out.println("8: Unblock Card");
    System.out.println("9: Apply for Loan");
    System.out.println("0: Exit to Main Menu");
```

for loop

- **Account Search**: The for loop iterates through the accounts array to find a specific user by account number, enabling transaction processing.

```
case 2: // ATM operations
    System.out.print("Enter your account number: ");
    accountNumber = scanner.nextDouble();
    OnlineATMSystem currentAccount = null;
    for (int i = 0; i < accountCount; i++) {
        if (accounts[i].getAccountNumber() == accountNumber) {
            currentAccount = accounts[i];
            break;
        }
    }
```

- **Transaction History**: Another for loop displays each entry in a user's transaction history when requested, providing a sequential view of past activities.

```
case 6:
    System.out.println("Transaction History:");
    for (String transaction : currentAccount.getTransactionHistory()) {
        System.out.println(transaction);
    }
    if (!currentAccount.askToContinue(scanner)) {
        break;
    }
    continue;
```

5. **Arrays**

- This array is used to store up to 100 OnlineATMSystem account objects. Each account created is added to this array, allowing the system to keep track of all the user accounts.

```
OnlineATMSystem[] accounts = new OnlineATMSystem[100]; // Store accounts
```

- This is a dynamic array (ArrayList) that stores unique PINs of all accounts created. It helps ensure that each account has a unique PIN by checking if a new PIN already exists in this list. ArrayLists are resizable and provide more flexibility compared to regular arrays in Java.

```
ArrayList<Integer> usedPins = new ArrayList<>(); // List to store unique PINs
```

6. **Classes**

Classes allows the program to create reusable, modular pieces of code where account-related data and behaviors are grouped together. This structure mirrors real-world banking systems, where different account types share common features but also have unique characteristics.

- The system is designed using the BankAccount abstract class, which is extended by OnlineATMSystem.
- This structure provides flexibility by separating common account behavior (e.g., withdrawal, deposit) from specific system implementations.

7. **Objects**

- Each account is represented as an object of the OnlineATMSystem class.
- For each new user, a new object is instantiated with specific details, like account holder name, city, balance, and PIN.
- These objects are stored in an array, which allows the program to track multiple accounts at the same time.

```
// Constructor (Inheritance)
public OnlineATMSystem(double accountNumber, String firstName, String lastName, String city, double initialBalance, int pin)
```

8. **Fields(Attributes)**

- Fields like balance, accountNumber, and transactionHistory store account-specific data.
- These fields are protected to allow access within subclasses but prevent unauthorized access from external code, maintaining data integrity.

```
// Abstract base class (Abstraction)
abstract class BankAccount {
    protected double accountNumber;
    protected String firstName;
    protected String lastName;
    protected String city;
    protected double balance;
    protected int pin;
    protected boolean isCardBlocked;
    protected ArrayList<String> transactionHistory;
    protected ArrayList<String> personalizedAlerts;
```

9. **Constructor**

- Constructors are used in the BankAccount class (and its subclasses) to initialize new objects with specific data. When a new account is created, the constructor assigns values to the

account's fields (e.g. firstName, lastName, City). Constructors ensure that each object is properly initialized with valid data before it can be used in the system.

```
// Constructor
public BankAccount(double accountNumber, String firstName, String lastName, String city, double initialBalance, int pin) {
    this.accountNumber = accountNumber;
    this.firstName = firstName;
    this.lastName = lastName;
    this.city = city;
    this.balance = initialBalance;
    this.pin = pin;
    this.isCardBlocked = false;
    this.transactionHistory = new ArrayList<>();
    this.personalizedAlerts = new ArrayList<>();
}
```

- The OnlineATMSystem constructor is used when creating an OnlineATMSystem object. It calls the parent BankAccount class's constructor, passing all necessary parameters. This initializes fields denined in BankAccount, like accountNumber and balance.

```
// Constructor (Inheritance)
public OnlineATMSystem(double accountNumber, String firstName, String lastName, String city, double initialBalance, int pin) {
    super(accountNumber, firstName, lastName, city, initialBalance, pin); // Call to parent class constructor
}
```

10. Methods (Accessor, Mutator)

- **Accessors**: Methods like getAccountNumber(), getBalance(), getPin(), getTranscationhistory(), getCity() allow controlled and safe access to private fields. This ensures that sensitive data, like the account number, can only be accessed through these methods, allowing the program to enforce rules on how data is retrieved.

```
public double getAccountNumber() {
    return accountNumber;
}
public double getBalance() {
    return balance;
}
public int getPin() {
    return pin;
}
public ArrayList<String> getTransactionHistory() {
    return transactionHistory;
}
public String getCity() {
    return city;
}
public boolean isCardBlocked() {
    return isCardBlocked;
}
public ArrayList<String> getPersonalizedAlerts() {
    return personalizedAlerts;
}
public void addAlert(String alert) {
    personalizedAlerts.add(alert);
}
```

- **Mutators**: Methods like deposit(), withdraw(), and transfer() allow modifications to the account's balance.

```
public void changePin(int newPin) {
    this.pin = newPin;
    System.out.println("PIN successfully changed.");
}
```

11. Parameters

- **double accountNumber**: Represents the account number for the bank account.
- **String firstName**: Represents the first name of the account holder.
- **String lastName**: Represents the last name of the account holder.
- **String city**: Represents the city where the account holder resides.
- **double initialBalance**: Represents the initial balance to be set for the account.
- **int pin**: Represents a unique Personal Identification Number (PIN) for the account.
- **boolean isCardBlocked**: Indicates whether the account holder's card is blocked.
- **ArrayList<String> transactionHistory**: Stores the history of transactions for the account.
- **ArrayList<String> personalizedAlerts**: Stores alerts related to the account.

12. Assignment

```
    this.accountNumber = accountNumber;
    this.firstName = firstName;
    this.lastName = lastName;
    this.city = city;
    this.balance = initialBalance;
    this.pin = pin;
    this.isCardBlocked = false;
    this.transactionHistory = new ArrayList<>();
    this.personalizedAlerts = new ArrayList<>();
}
```

13. Conditional Statements

Conditional statements are used to guide the program's flow based on user choices and data.

- If statement
- The if-else statements ensure whether the user's balance is sufficient before allowing the transaction to proceed or not.
- The switch statement is employed to provide users with a selection of operations, such as withdrawal, deposit, transfer, and more. This helps streamline the flow of options and makes the code more readable when there are multiple choices. For instance, after verifying the account number and PIN, a switch statement presents the user with various ATM operations.

14. (Scanner) / Input Handling

- The Scanner class handles user input, reading data from the keyboard. It is used throughout the program to gather necessary information (e.g., name, balance, pin).
- Input handling is critical in interactive programs like this, as it allows users to provide data in real-time.
- Without input handling, the program wouldn't be able to respond to user actions, making the entire system static.

15. Polymorphism

Method Overriding

Polymorphism is demonstrated through method overriding, used in the OnlineATMSystem class to override the abstract methods from BankAccount, providing specific implementations for withdrawal and deposit functionality.

→ **withdraw**: Checks if the balance is sufficient for the withdrawal amount. If so, it deducts the amount, logs the transaction, and displays the remaining balance; otherwise, it shows an "Insufficient balance" message.

→ **deposit**: Increases the balance by the deposited amount, logs the transaction, and displays the updated balance.

```
// Overriding the withdraw method (polymorphism)
@Override
public void withdraw(double amount) {
    if (amount <= balance) {
        balance -= amount;
        transactionHistory.add("Withdraw: Rs" + amount);
        System.out.println("Please collect your cash: Rs" + amount);
        System.out.println("Remaining balance: Rs" + balance);
    } else {
        System.out.println("Insufficient balance.");
    }
}

// Overriding the deposit method (polymorphism)
@Override
public void deposit(double amount) {
    balance += amount;
    transactionHistory.add("Deposit: Rs" + amount);
    System.out.println("Amount Deposited: Rs" + amount);
    System.out.println("Updated balance: Rs" + balance);
}
```

16. Inheritance

- The OnlineATMSystem class inherits from BankAccount, allowing it to reuse fields and methods defined in the parent class.
- Inheritance reduces code duplication and makes the program easier to maintain by sharing common code across related classes.

```
// OnlineATMSystem inherits BankAccount (Inheritance)
public class OnlineATMSystem extends BankAccount {
```

17. Abstraction

Abstraction is achieved through the BankAccount class, which provides a high-level blueprint for accounts, including shared fields and methods. It leaves specific details of how certain operations (like deposit and withdraw) should be implemented to the subclasses.

```
// Abstract methods (Abstraction)
public abstract void withdraw(double amount);
public abstract void deposit(double amount);
```

18. Encapsulation

- Encapsulation ensures that sensitive data, such as the account balance and PIN, are hidden and only accessible or modifiable through specific methods like deposit() and withdraw()
- This protects the integrity of the account and prevents external code from making unauthorized changes.

```
protected double accountNumber;
protected String firstName;
protected String lastName;
protected String city;
protected double balance;
protected int pin;
protected boolean isCardBlocked;
protected ArrayList<String> transactionHistory;
protected ArrayList<String> personalizedAlerts;
```

Conclusion

The Online ATM System successfully implements key object-oriented principles such as encapsulation, inheritance, polymorphism to manage account operations securely and efficiently. Its modular design and easy-to-use interface provide a seamless user experience, ensuring secure and flexible management of accounts, different transactions, PINs etc. The system successfully simulates real-world ATM functions, offering scalability and maintainability for future enhancements.